Technical documentary

Virtual Tape Measure System

1. System Overview

The **Virtual Tape Measure** is a real-time computer vision application designed to dynamically detect, track, and measure the physical dimensions of objects placed within a camera's field of view. It utilizes a dynamic background subtraction model combined with adaptive thresholding to isolate foreground objects, ensuring high accuracy regardless of ambient lighting conditions. The system features a built-in calibration module utilizing a standard CR80 credit card to dynamically calculate the pixel-to-centimeter ratio.

2. Dependencies

To run this software, the following Python libraries are required:

- `cv2` (OpenCV): Core computer vision operations, matrix manipulation, and hardware interfacing.
- `numpy` (as `np`): High-performance mathematical operations, specifically for coordinate transformations and median calculations.

3. Global Configuration (Constants)

The system's behavior is governed by several hyper-parameters defined at the top of the script:

- `CARD_WIDTH_CM` **(8.56) /** `CARD_HEIGHT_CM` **(5.398):** The exact ISO/IEC 7810 ID-1 standard dimensions of a bank card, used as the absolute ground truth for calibration.
- `PIXELS_PER_CM_DEFAULT` **(35.0):** The fallback conversion ratio before user calibration.
- `MIN_CONTOUR_AREA` **(3000):** The absolute minimum pixel footprint an object must have to be processed. This acts as a primary noise filter against dust or minor

shadows.

- `SMOOTHING_FRAMES` **(20):** The buffer size for the moving median filter. It stores the dimensions from the last 20 frames to eliminate flickering and provide stable output.

4. Architecture & Core Pipeline (`VirtualTapeMeasure` Class)

4.1. Initialization (`__init__`)

- **Hardware Setup:** Initializes the camera (`VideoCapture(0)`) and explicitly requests a `1280x720` (720p) resolution to ensure sufficient pixel density for accurate measurement.
- **State Management:** Sets up boolean flags (`self.calibrating`) and data buffers (`self.history_w`, `self.history_h`).
- **MOG2 Background Subtractor:** Initializes `cv2.createBackgroundSubtractorMOG2(history=100, varThreshold=50, detectShadows=False)`. This algorithm continuously learns the background (e.g., an empty desk) so it can instantly recognize when a new object is introduced.
- **Warm-up Phase:** Enforces a 60-frame delay before processing measurements to allow the camera's auto-exposure and the MOG2 subtractor to stabilize.

4.2. Object Detection Pipeline (`find_objects`)

This method handles the raw image processing. It fuses two distinct techniques to create a highly robust foreground mask:

1. **Adaptive Thresholding:** The frame is converted to grayscale and blurred. `cv2.adaptiveThreshold` is applied. Unlike a global threshold, this calculates the threshold for small regions (`blockSize=25`), making the system highly resilient to uneven lighting or shadows across the desk.
2. **Background Subtraction:** The MOG2 algorithm is applied to extract moving or newly placed foreground objects.

3. **Mask Fusion:** The adaptive threshold mask and the MOG2 mask are merged using a bitwise OR operation (`cv2.bitwise_or`).
4. **Morphological Cleanup:**
   - *Closing (`MORPH_CLOSE`):* Fills in small black holes inside the white object mask.
   - *Opening (`MORPH_OPEN`):* Erases tiny white noise specks outside the object.
5. **Contour Extraction:** Extracts the outer boundary coordinates (`findContours`) of the resulting clean shapes.

4.3. Geometric Filtering (`filter_contours`)

Not all detected contours are valid objects. This method applies a rigorous validation checklist:

- **Area Check:** Discards contours smaller than `MIN_CONTOUR_AREA`.
- **Boundary Margin Check:** Rejects objects touching the edges of the camera frame (defined by a 15-pixel `margin`). If an object is cut off by the screen edge, its measured dimensions will be mathematically incorrect.
- **Solidity Check:** Calculates the contour's Convex Hull. Solidity is the ratio of the contour area to its convex hull area. If the ratio is `< 0.4`, the shape is highly irregular (like a tangled cable or a hand) and is discarded to prevent erratic bounding boxes.

4.4. Mathematical Core (`draw_and_measure`)

This method isolates the largest valid contour and executes the mathematical transformations.

1. **Bounding Box Calculation:** Uses `cv2.minAreaRect` to generate the smallest possible rotated rectangle enclosing the object. This ensures accurate measurement even if the object is placed diagonally.
2. **Calibration Mode:** If `self.calibrating` is `True`, it forces the system to assume the current object is the reference bank card. It divides the pixel dimensions by `CARD_WIDTH_CM` and `CARD_HEIGHT_CM` to establish a new, highly accurate `pixels_per_cm` ratio.

3. **Measurement Mode:** * Converts current pixel dimensions to centimeters.
    - Appends the raw data to `history_w` and `history_h`.
    - Applies a **Median Filter** (`np.median`). Unlike a simple average, the median completely ignores sudden, extreme miscalculations (outliers) caused by glare or hand movement, resulting in rock-solid numbers on the UI.
4. **Rendering:** Calls helper functions to draw the green bounding box and dimension overlays.

4.5. User Interface Renderers

- `_draw_dimension_lines`**:** A UI enhancement that calculates the midpoints of the bounding box edges and draws CAD-style reference lines and center points (`cv2.line`, `cv2.circle`).
- `draw_hud`**:** Renders the static text on the screen, including the current pixel-to-cm ratio, active keyboard commands, and the warm-up progress bar. It utilizes text drop-shadows (drawing black text slightly offset behind white/colored text) for readability on any background.

4.6. Main Execution Loop (`run`)

- Handles camera error state management.
- Manages the infinite `while True` loop that continuously grabs new frames (`self.cap.read()`).
- Listens for keyboard interrupt signals (`cv2.waitKey(1)`).
- Guarantees memory cleanup and camera release (`self.cap.release()`) within a `finally` block if the program crashes or exits.

5. Keyboard Controls Reference

Key	Action	Description
`ESC`	Exit	Safely terminates the application and releases the camera hardware.

`C`	Calibrate	Toggles Calibration Mode. The user must place a bank card on the surface, press `C`, ensure the blue box aligns, and press `C` again to save.
`R`	Reset	Purges the measurement history buffers and resets the conversion ratio to `PIXELS_PER_CM_DEFAULT`.